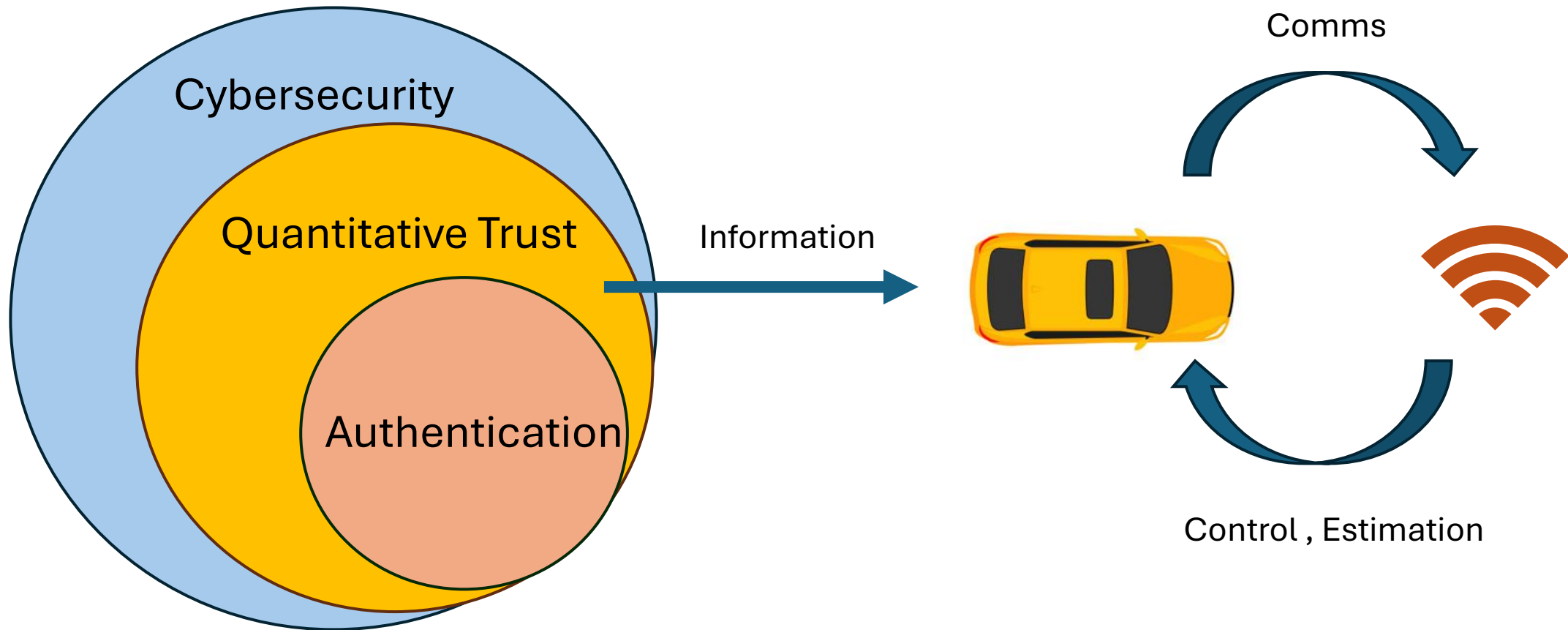


Quantitative Trust

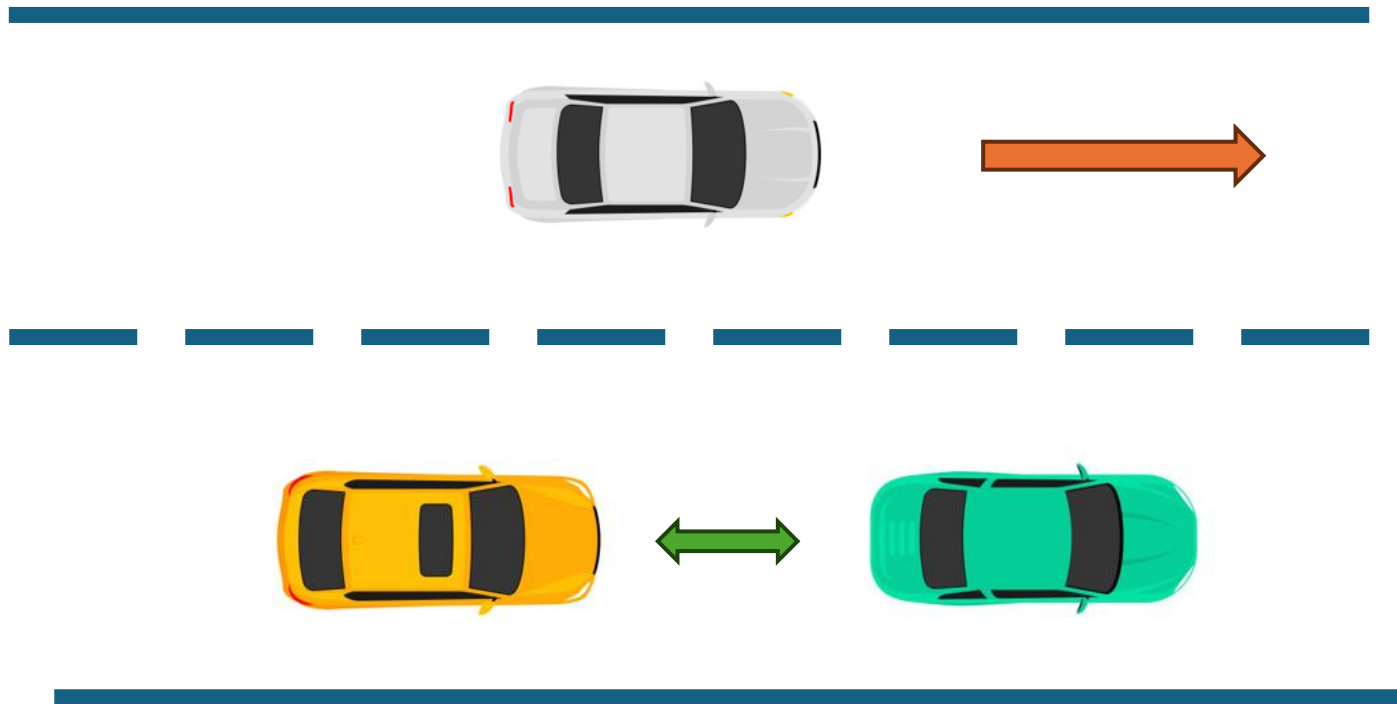


- ⚠ Problem: Wrong data → unsafe behavior
- ✓ Solution: Evaluate trust before using data.

Consensus

In control : Platooning on a road network agreement on velocity and heading

In estimation : Reach the same value on all the estimation state



State Definitions in Vehicle Platooning

- **Local State**

State of an individual vehicle (e.g., position, velocity, heading).

- **Global / Fleet / Distributed State**

Combined state of all vehicles in the platoon.

- **Absolute Position**

Position expressed in a world-fixed frame (GPS / map coordinates).

- **Relative Position**

Distance or offset between vehicles (vehicle-to-vehicle frame).

Dynamic Model of Single Vehicle

We consider a vehicle with Ackermann steering geometry, whose kinematics can be approximated by the bicycle model as follows:

$$\dot{s}_x = v \cos(\theta) \qquad \dot{s}_y = v \sin(\theta)$$

$$\dot{v} = a \qquad \dot{a} = \frac{1}{\tau}u - \frac{1}{\tau}a \qquad \dot{\theta} = \frac{v}{L} \tan(\delta)$$

- s_x , and s_y denote the coordinates of longitudinal and lateral locations of the vehicle position;
- v and a are the velocity and acceleration, respectively;
- δ is the steering angle;
- θ is the orientation (yaw) angle;
- u is the acceleration control signal;
- L is the vehicle's wheel base.

$$x_i = \begin{bmatrix} s_x & s_y & v & a & \theta \end{bmatrix}^\top$$

Dynamic Model of Single Vehicle

We consider a platoon comprised of N vehicles ,

each of the vehicles in the platoon has the following state-space equation:

$$\begin{cases} x_i(t+1) = A_b x_i(t) + B_b u_i(t) + \Delta_i(t), \\ y_i(t) = C_b x_i(t), \end{cases} \quad i \in \mathcal{V},$$

where $\mathcal{V} = \{1, 2, \dots, N\}$ is the set of vehicle.

Each vehicle has a full measurement state obtained in real time, including the position, and velocity, orientation angle from a GPS, IMU sensor mounted on the vehicle.

Local Observer

$$\hat{x}_0^{(i)}(t+1) = A_b \hat{x}_0^{(i)}(t) + B_b u_i(t) + F_i(y_i(t) - C_b \hat{x}_0^{(i)}(t))$$

Distributed Observer

$$\hat{x}_i^{(j)}(t+1) = A_b \left(\hat{x}_i^{(j)}(t) + \sum_{l \in \mathcal{N}_i} w_{il}^{(j)}(t) (\hat{x}_l^{(j)}(t) - \hat{x}_i^{(j)}(t)) + w_{i0}^{(j)}(t) (\hat{x}_0^{(j)}(t) - \hat{x}_i^{(j)}(t)) \right) + B_b \hat{u}_j(t)$$

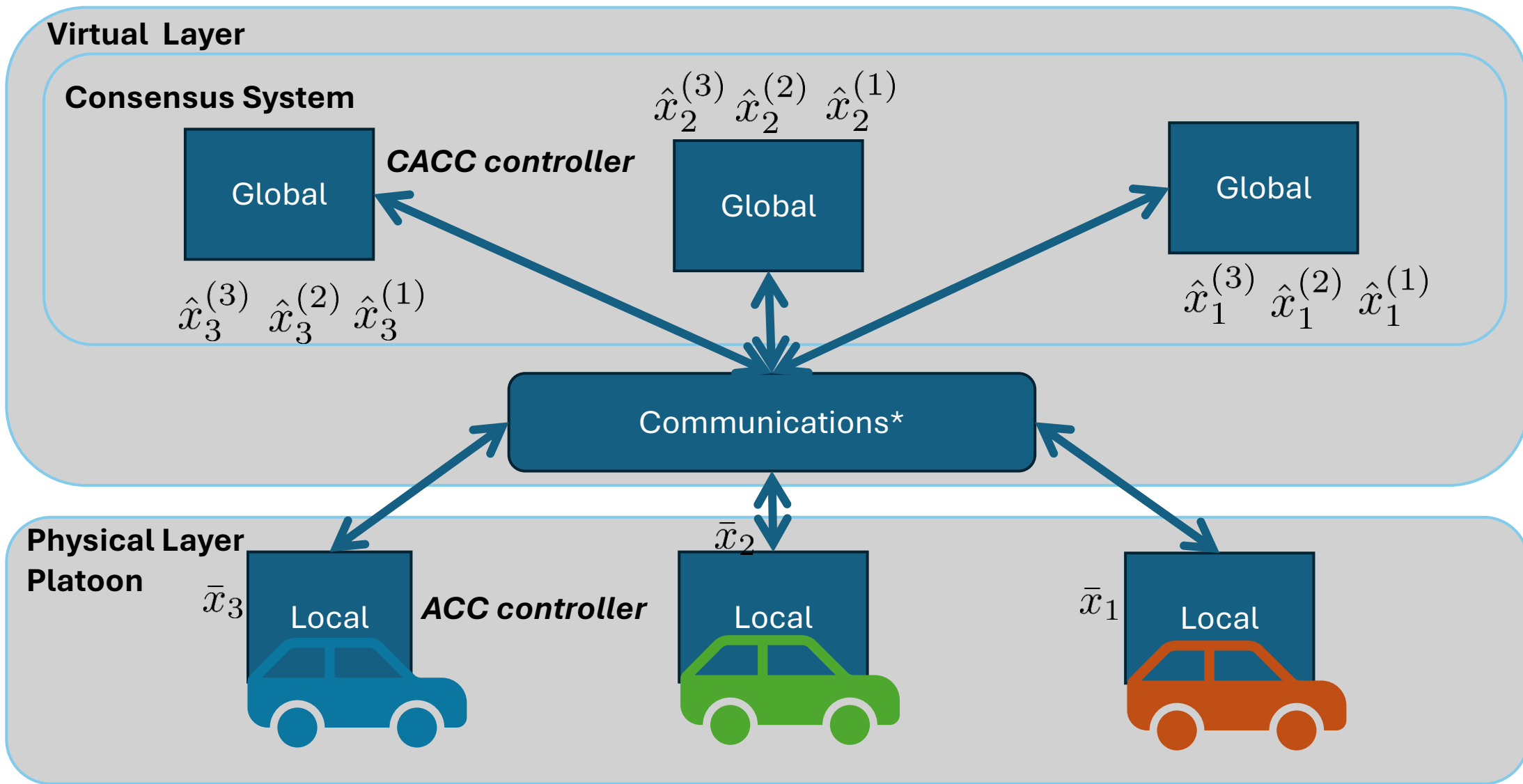
or

$$\hat{x}_i^{(j)}(t+1) = A_b \left(w_{ii}^{(j)}(t) \hat{x}_i^{(j)}(t) + \sum_{l \in \mathcal{N}_i} w_{il}^{(j)}(t) \hat{x}_l^{(j)}(t) + w_{i0}^{(j)}(t) \hat{x}_0^{(j)}(t) \right) + B_b \hat{u}_j(t)$$

Where :

$w_{il}^{(j)}$ and $w_{i0}^{(j)}$: are nonnegative consensus weights related to the communication network

$w_{ii}^{(j)}(t) = 1 - w_{i0}^{(j)}(t) - \sum_{l \in \mathcal{N}_i^{(j)}(t)} w_{il}^{(j)}(t)$: is residual self-weight



Exemple :

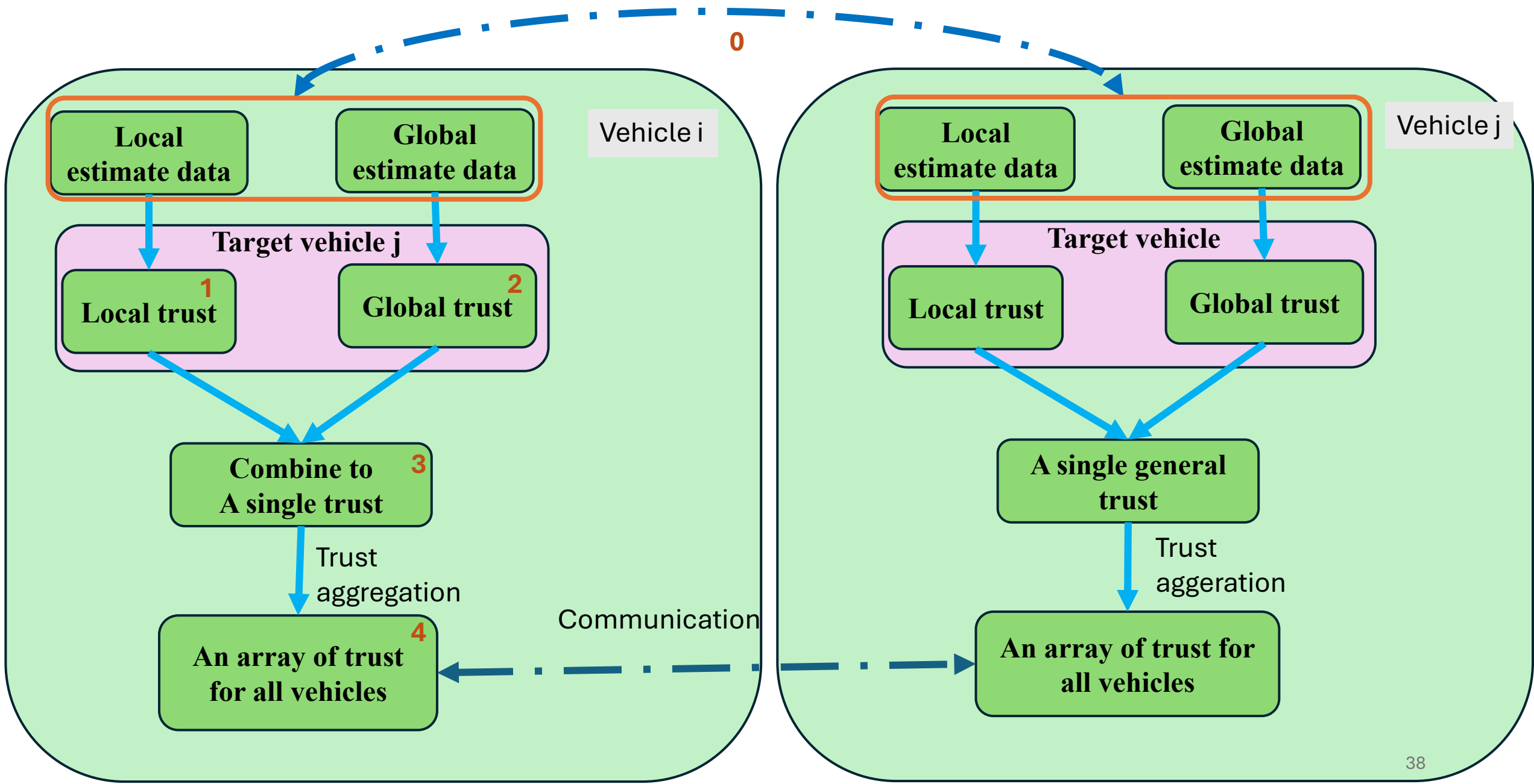
- $\hat{x}_2^{(1)}$ Estimate value of vehicle 2 in vehicle 1
- $\bar{x}_2$ Estimate local value of vehicle 2

*Communication protocole : P2P , Host/Server-Client

How we build the Trust ?

- ❑ Its speed doesn't conflict with what others report
- ❑ Its distance makes physical sense
- ❑ Its acceleration follows real-world behavior
- ❑ Its distributed estimate matches the host's estimate
- ❑ Its messages are fresh and authenticated

Trust Score Framework



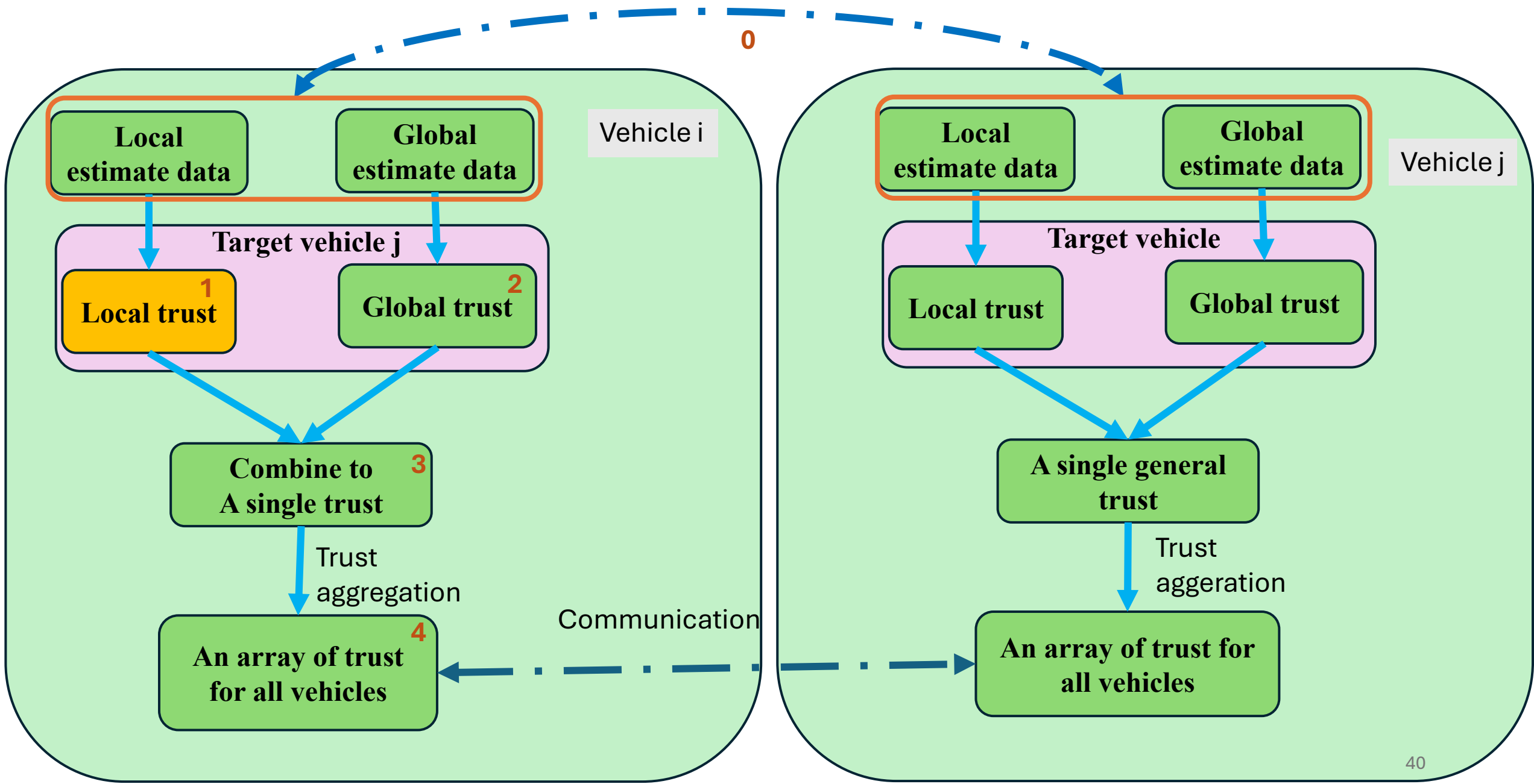
0. Data Validity Indicator

- Prevents **spoofing** (fake vehicle pretending to be another one).
- Prevents **replay attacks** (old packets being resent by an attacker).

$$\delta_{i,l}(t) = \begin{cases} 1, & \text{if message is authenticated and fresh,} \\ 0, & \text{otherwise.} \end{cases}$$

Only packets with $\delta_{i,l}(t) = 1$ are used in the trust evaluation

Trust Score Framework



1. Local Trust (LT)

Checks physical consistency:

- Velocity mismatch
- Distance mismatch
- Acceleration mismatch
- Heading mismatch

$$\mathcal{LT}_{i,l}(t) = \begin{cases} f(\iota_{i,l}^{\text{velocity}}, \iota_{i,l}^{\text{distance}}, \iota_{i,l}^{\text{accel}} \dots) & \text{if } \delta_{i,l}(t) = 1 \\ (1 - \lambda_h) \mathcal{LT}_{i,l}(t - 1), & \text{otherwise} \end{cases}$$

1. Local Trust (LT)

Example :

$$\mathcal{LT}_{i,l}(t) = \begin{cases} f(\iota_{i,l}^{\text{velocity}}, \iota_{i,l}^{\text{distance}}, \iota_{i,l}^{\text{accel}} \dots) & \text{if } \delta_{i,l}(t) = 1 \\ (1 - \lambda_h) \mathcal{LT}_{i,l}(t - 1), & \text{otherwise} \end{cases}$$

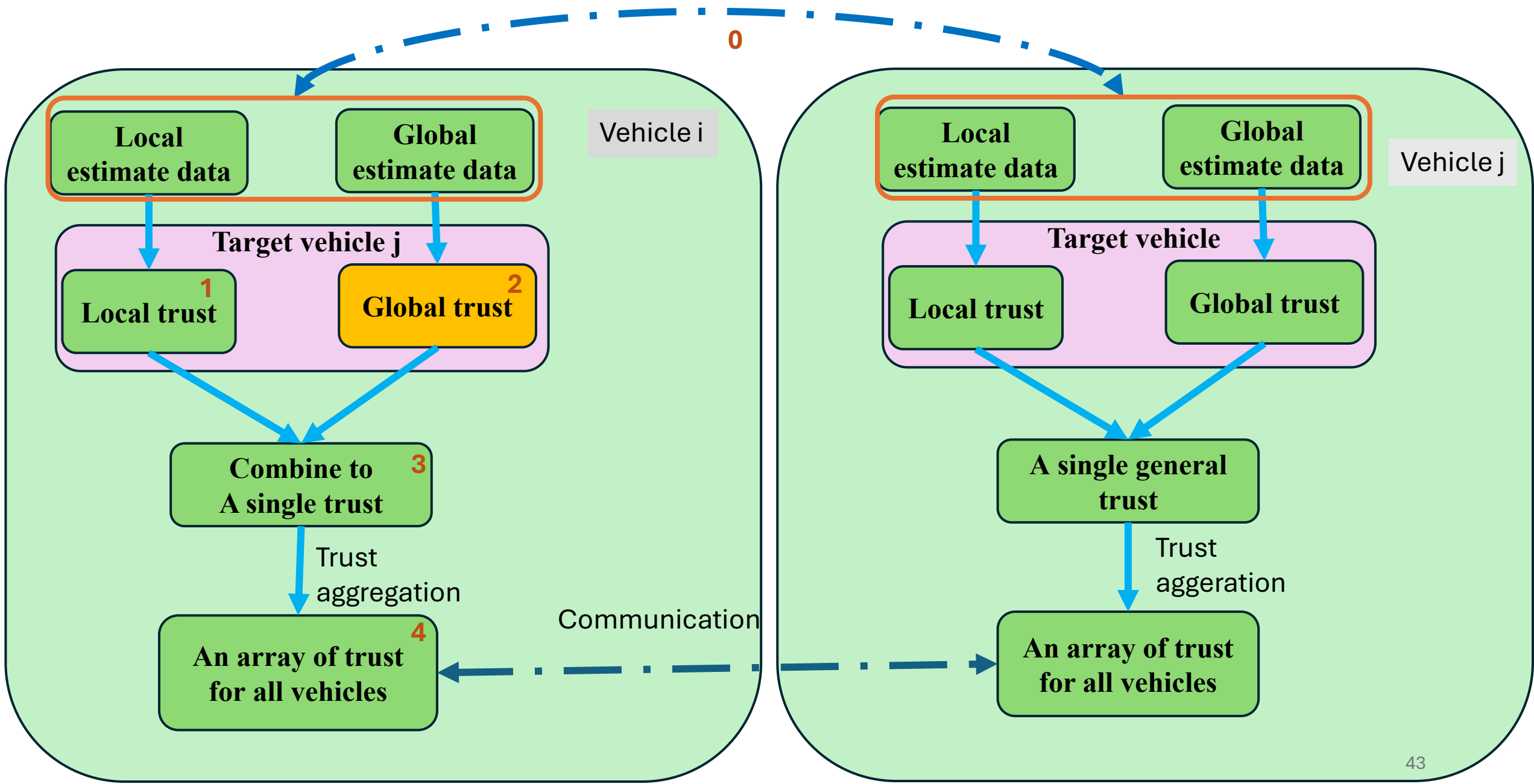
Velocity mismatch

$$\iota_{i,l}^{\text{velocity}} = \begin{cases} \max \left(1 - \left| \frac{\hat{v}_0^{(l)} - \hat{v}_0^{(i)}}{\hat{v}_0^{(i)}} \right|, 0 \right), & \text{if } \hat{v}_0^{(i)} > 0, \\ \max \left(1 - |\hat{v}_0^{(l)}|, 0 \right), & \text{otherwise,} \end{cases}$$

Distance mismatch

$$d_{i,l} = \hat{s}_{x,0}^{(l)} - \hat{s}_{x,0}^{(i)} - L$$
$$\iota_{i,l}^{\text{distance}} = \max \left(1 - \left| \frac{d_{i,l} - d}{d} \right|, 0 \right)$$

Trust Score Framework



2. Global/Distributed Trust (DT)

Evaluates whether the neighbor's distributed estimate is globally believable

Three score is use that coming from :
host estimate, local measurement, self-consistency.

$$\mathcal{DT}_{i,l} = \begin{cases} \gamma_{i,l}^{\text{self}} \cdot \gamma_{i,l}^{\text{local}} \cdot \gamma_{i,l}^{\text{host}} & \text{if } \delta_{i,l}(t) = 1 \\ (1 - \lambda_h) \mathcal{DT}_{i,l}(t - 1), & \text{otherwise} \end{cases}$$

2. Global/Distributed Trust (DT)

$$\mathcal{DT}_{i,l} = \begin{cases} \gamma_{i,l}^{\text{self}} \cdot \gamma_{i,l}^{\text{local}} \cdot \gamma_{i,l}^{\text{host}} & \text{if } \delta_{i,l}(t) = 1 \\ (1 - \lambda_h) \mathcal{DT}_{i,l}(t-1), & \text{otherwise} \end{cases}$$

Consistency with Host's Global Estimate

$$\gamma_{i,l}^{\text{host}}$$

The host compare received data directly to its own fleet estimate

1. We use the **Mahalanobis distance**, which normalizes discrepancies based on covariance: to get a total discrepancy score:

$$D_{i,l}(k) = \sum_{j=1}^n \left(\hat{\mathbf{x}}_i^{(j)}(k) - \hat{\mathbf{x}}_l^{(j)}(k) \right)^T \Sigma^{-1} \left(\hat{\mathbf{x}}_i^{(j)}(k) - \hat{\mathbf{x}}_l^{(j)}(k) \right)$$

where: **The covariance matrix** is diagonal:

$$\Sigma = \text{diag}(\sigma_p^2, \sigma_v^2, \sigma_a^2)$$

A small $D_{i,l}(k)$ means l 's global estimate is close to the host's, suggesting higher trustworthiness.

2. **Convert to a Trust Factor** Turn this discrepancy into a trust score using an exponential function:

$$\gamma_{i,l}^{\text{host}}(k) = \exp(-D_{i,l}(k))$$

The result, $\gamma_{i,l}^{\text{host}}(k)$, ranges from 0 to 1. A value near 1 means high trust (small discrepancy), while a value near 0 means low trust (large discrepancy).

2. Global/Distributed Trust (DT)

$$\mathcal{DT}_{i,l} = \begin{cases} \gamma_{i,l}^{\text{self}} \cdot \gamma_{i,l}^{\text{local}} \cdot \gamma_{i,l}^{\text{host}} & \text{if } \delta_{i,l}(t) = 1 \\ (1 - \lambda_h) \mathcal{DT}_{i,l}(t-1), & \text{otherwise} \end{cases}$$

Consistency with Local Sensor Measurements

$\gamma_{i,l}^{\text{local}}$

Ensure that fleet estimates received from neighbors vehicles align with trusted local relative measurement.

$$\varepsilon_{i,l}^{(j)} = H(\hat{x}_l^{(j)} - \hat{x}_l^{(i)}) - y_{i,j}, \quad E_{i,l} = \sum_{j \in \mathcal{N}_i} \varepsilon_{i,l}^{(j)\top} \Sigma_{\text{local}}^{-1} \varepsilon_{i,l}^{(j)}$$

$$\gamma_{i,l}^{\text{local}} = \exp(-E_{i,l})$$

$y_{i,j}$: Relative distance , velocity

2. Global/Distributed Trust (DT)

$$\mathcal{DT}_{i,l} = \begin{cases} \gamma_{i,l}^{\text{self}} \cdot \gamma_{i,l}^{\text{local}} \cdot \gamma_{i,l}^{\text{host}} & \text{if } \delta_{i,l}(t) = 1 \\ (1 - \lambda_h) \mathcal{DT}_{i,l}(t-1), & \text{otherwise} \end{cases}$$

Self-Consistency with Local Sensor Measurements

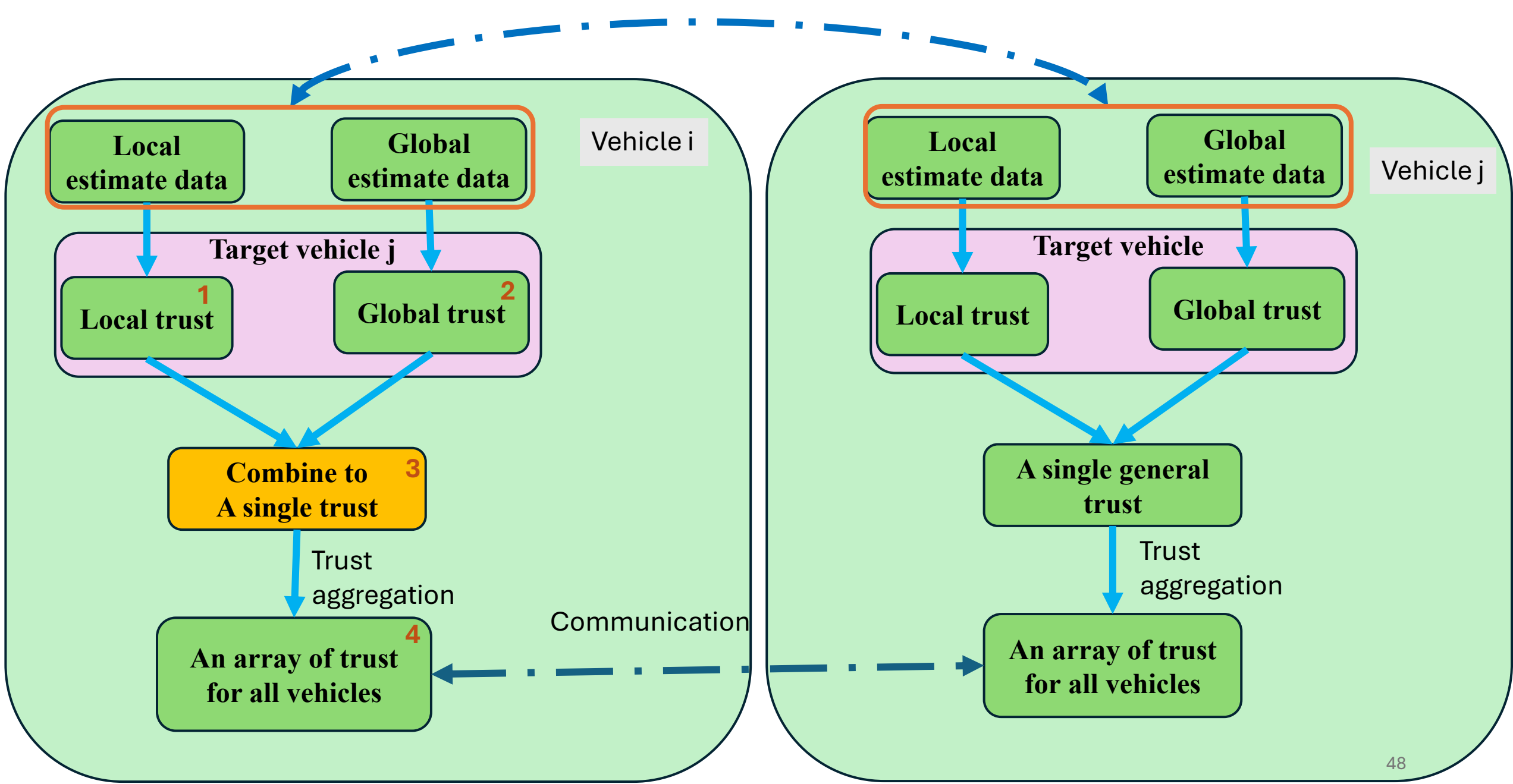
$$\gamma_{i,l}^{\text{self}}$$

Ensure that host fleet estimates align with trusted local sensor data.

$$\varepsilon_{i,\text{self}}^{(j)} = H(\hat{x}_i^{(j)} - \hat{x}_i^{(i)}) - y_{i,j}, \quad E_{i,\text{self}} = \sum_{j \in \mathcal{N}_i} \varepsilon_{i,\text{self}}^{(j)\top} \Sigma_{\text{self}}^{-1} \varepsilon_{i,\text{self}}^{(j)}$$

$$\gamma_{i,l}^{\text{self}} = \exp(-E_{i,\text{self}})$$

Trust Score Framework



3. Vehicle Trust (VT)

Trust at time t :

$$\mathcal{T}_{i,l}(t) = \mathcal{L}\mathcal{T}_{i,l} \cdot \mathcal{D}\mathcal{T}_{i,l}.$$

Target trust is the instantaneous score at time t

3. Vehicle Trust (VT)

$$\text{Trust at time } t: \mathcal{T}_{i,l}(t) = \mathcal{L}\mathcal{T}_{i,l} \cdot \mathcal{D}\mathcal{T}_{i,l}.$$

Trust smoothing over time

- We classify trust into 5 levels:
Unreliable – Poor – Acceptable – Good – Excellent
- Recent data affects trust more strongly.
- Old data slowly fades away.
- This prevents trust from changing too fast due to noise or brief packet losses.

Step smoothed trust:

1. Trust history :
$$R_{i,l}(t+1) = (1 - \mathcal{T}_{i,l}(t) \cdot \beta) \cdot R_{i,l}(t) + r_{i,l}(t),$$

2. Trust distribution :

$$S_{i,l}(k) = \frac{R_{i,l}(k) + C \cdot \alpha(k)}{C + \sum_{j=1}^q R_{i,l}(j)},$$

3. Expected smooth trust

$$\mathcal{V}\mathcal{T}_{i,l} = \sum_{j=1}^q \frac{j-1}{q-1} \cdot S_{i,l}(j).$$

3. Vehicle Trust (VT)

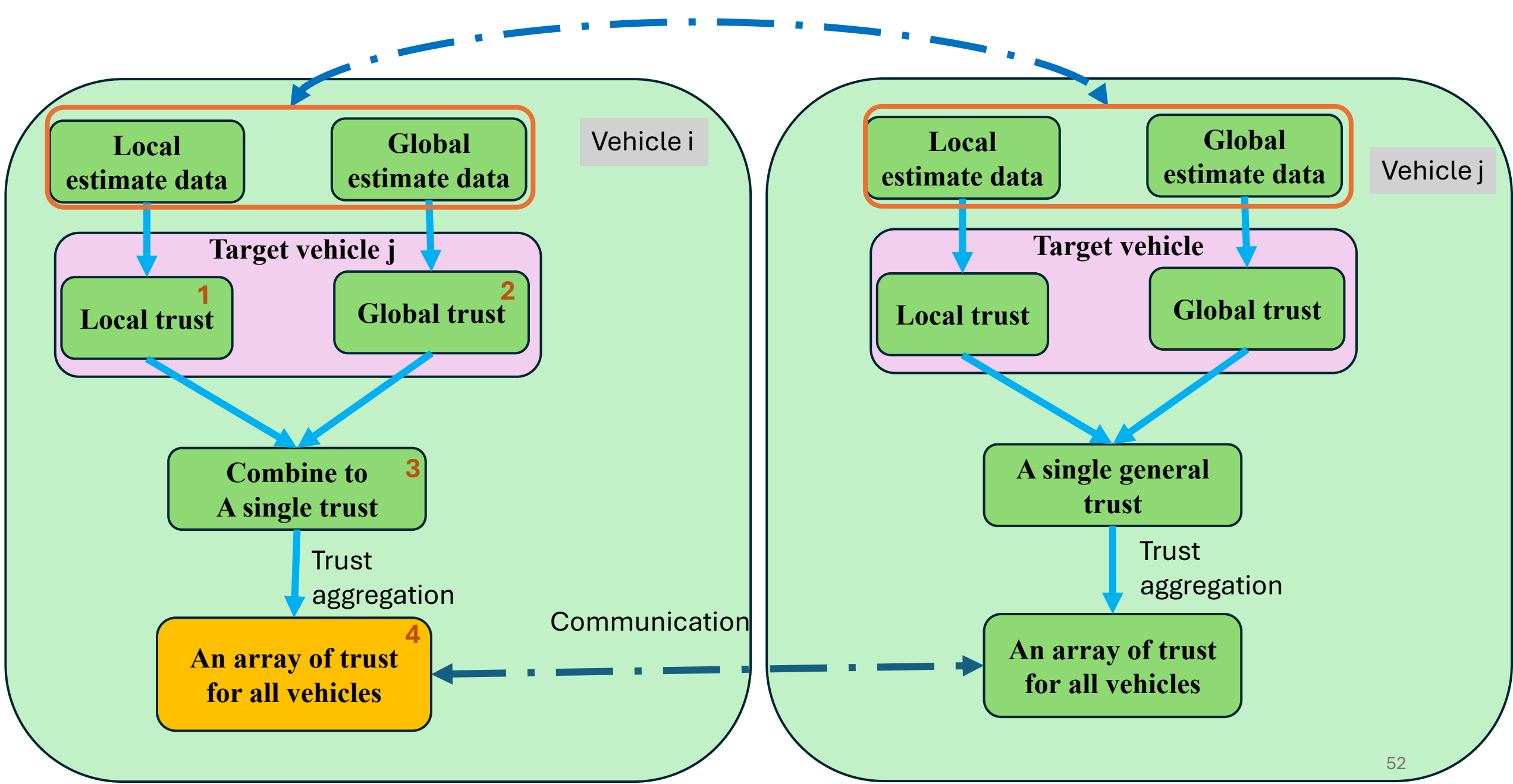
An example the trust distribution $S_y = (0.134, 0.197, 0.329, 0.263, 0.076)$:

$$\mathcal{VT}_{i,l} = \sum_{k=1}^5 \frac{k-1}{4} \cdot S_y(k)$$

$$\begin{aligned}\mathcal{VT}_{i,l} &= (0 \cdot 0.134) + (0.25 \cdot 0.197) + (0.5 \cdot 0.329) + (0.75 \cdot 0.263) + (1 \cdot 0.076) \\ &= 0 + 0.04925 + 0.1645 + 0.19725 + 0.076 = 0.487\end{aligned}$$

So, $\mathcal{VT}_{i,l} \approx 0.49$, which is close to "acceptable" (0.5).

Trust Score Framework



4. Generalized Trust Vector

Purpose: Each vehicle i obtains a trust opinion about ALL vehicles.

$$O_i(j) = \begin{cases} 1, & j = i, \\ VT_{i,j}, & j \in \mathcal{N}_i, \\ \text{wMed}(\{O_k(j)\}, \{w_{i,k}\}), & NS_{i,j} \neq \emptyset, \\ \frac{\sum_{k \in \mathcal{N}_i} g_{i,k} O_k(j)}{\sum_{k \in \mathcal{N}_i} g_{i,k}}, & NS_{i,j} = \emptyset. \end{cases}$$

- Self-trust
- Direct-neighbor trust
- One-hop trust propagation
- Fallback rule when data missing

$NS_{i,j}$ is the group of trusted direct neighbors of i that can provide an opinion about vehicle j .

Using Trust for Weight Design in the Observer

We redefine a time-dependent trusted neighborhood for each agent i as :

$$\mathcal{LN}_i^{(j)}(t) = \left\{ l \in N_i \mid O_i(l, t) \geq \theta_{\min} \right\} \cup \left(\{0\} \text{ if } \mathcal{LT}_{i,j}(t) \geq \theta_{\min} \right)$$

This is the set of neighbors that legitimate agent i classifies as its legitimate neighbors at time t . Denote

$$n_{il}^{(j)}(t) = \left| \mathcal{LN}_i^{(j)}(t) \right| + 1, \forall l \in \mathcal{LN}_i^{(j)}$$

The weight matrix that priority the local estimation term is defined as

$$w_{il}^{(j)}(t) = \begin{cases} \frac{1}{n_{il}^{(j)}(t)}, & l \in \mathcal{LN}_i^{(j)}(t) \\ 0, & \text{otherwise.} \end{cases}$$

✓ Ensures resilient estimation

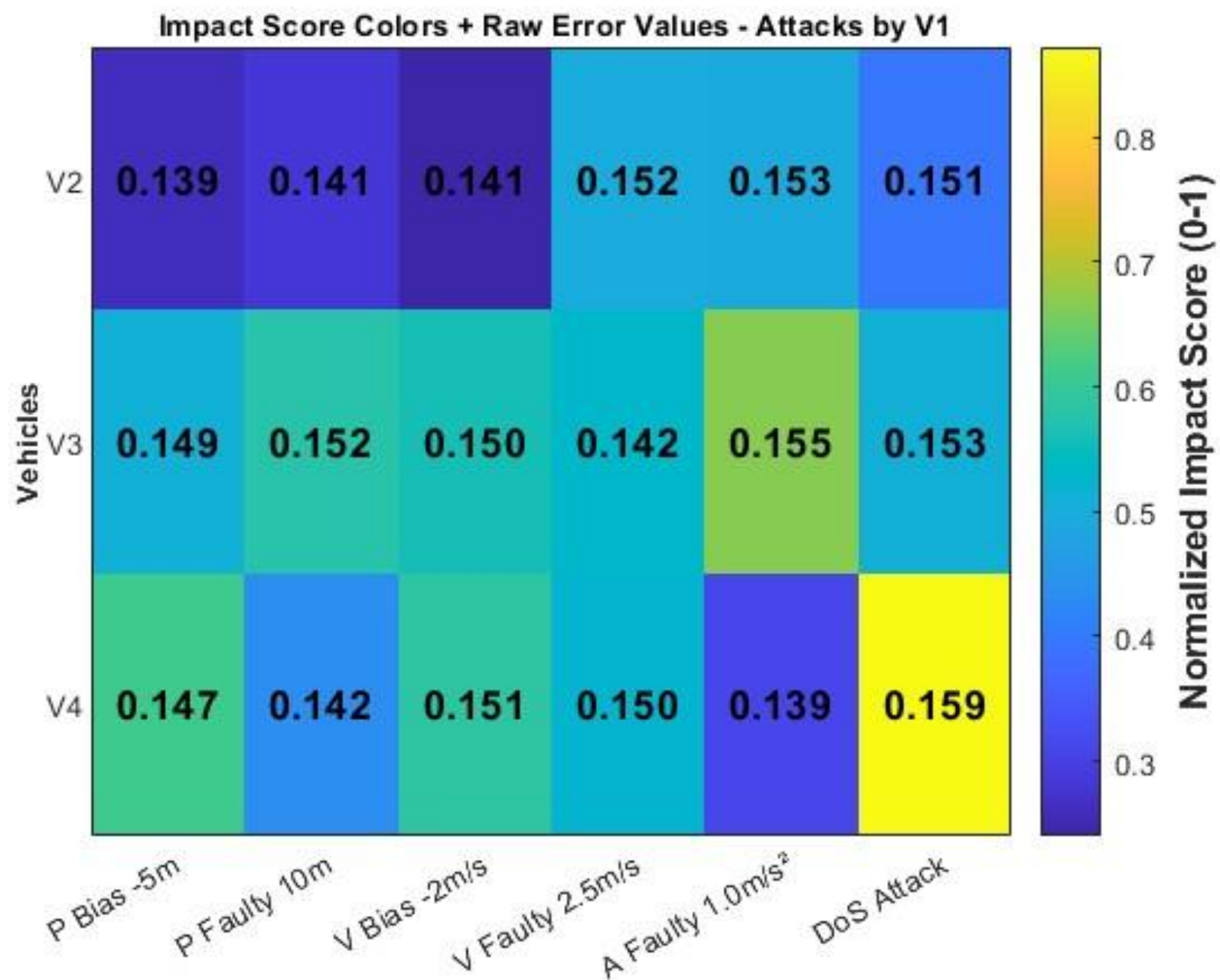
✓ Isolates malicious vehicles

Numerical Simulation

4 vehicle in platoon , communication V2V
Vehicle 1 is attacker send false data to the platoon
Attack interval start at 10 end at 15s

Case	Type	Target	Parameters
1	Bias	Position	bias = -5 m
2	Faulty	Position	int. = 10, p = 0.3
3	Bias	Velocity	bias = -2 m/s
4	Faulty	Velocity	int. = 2.5, p = 0.3
5	Faulty	Acceleration	int. = 1.0, p = 0.3
6	Drop	All	p_drop = 0.5

Attack Cases Table



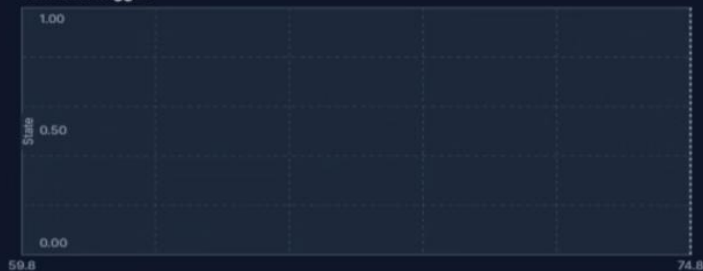
Direct and Generalized Trust



Final Weights for Target V0



Rollback Trigger



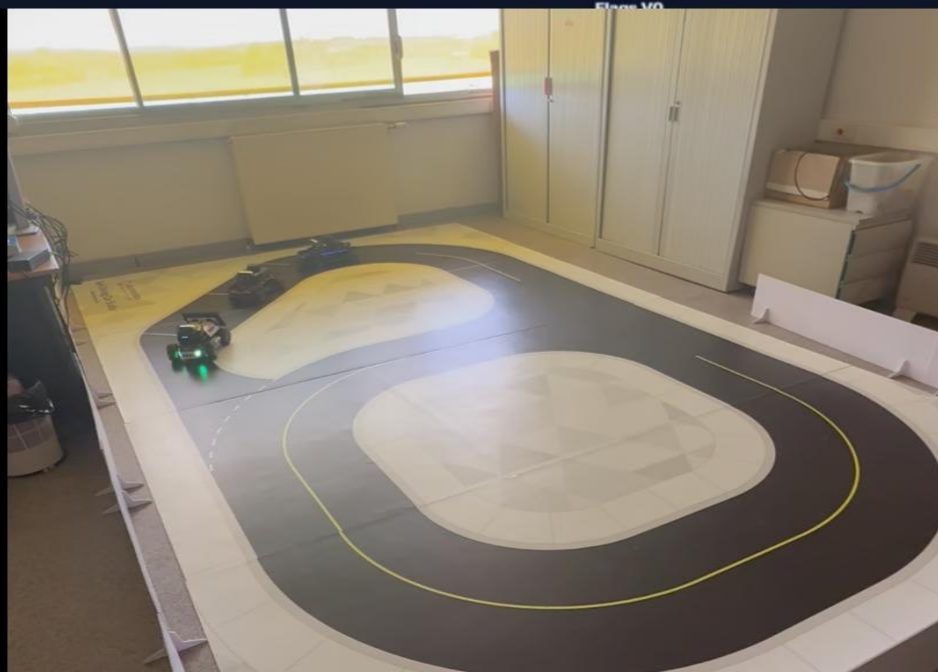
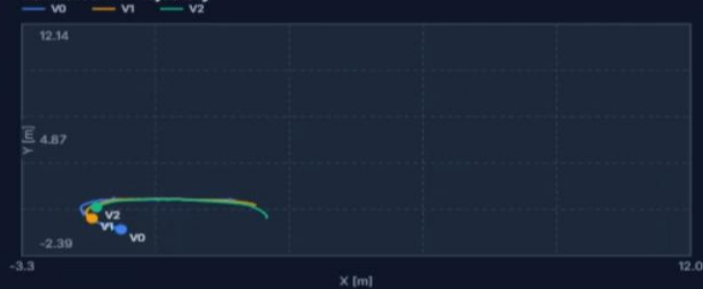
Component Scores Local Trust V0



Component Scores Global Trust V0



Estimated XY Trajectory



Estimated XY Trajectory



Roll back

The Case When Rollback Is Useful:

- Attacker V0 sends bad data (send Fake position).
- Honest neighbor V2 uses that bad data in its own estimator (trust detected late , V1 also).
- Later host vehicle V1 receives V2 fleet estimate of V0 .
- From V1 perspective, V2 is honest, but V2 estimate is already contaminated upstream.
- Even later V1 see V2 is malicious , V1 already contaminated , so the pure prediction will go wrong

Roll back

The Case When Rollback Is Useful:

- Attacker V0 sends bad data (send Fake position).
- Honest neighbor V2 uses that bad data in its own estimator (trust detected late).
- Later host vehicle V1 receives V2 fleet estimate of V0 .
- From V1 perspective, V2 is honest, but V2 estimate is already contaminated upstream.

Rollback is used when an attack is detected late.
We can have a good base initial state for prediction

Roll back

Repair step after the trust detector has already found bad contributors

At every time step k , before the observer update is finalized, we save the information needed to replay the update later.

$$\mathcal{B}(k) = \left\{ \hat{x}_i^{(j)}(k), \hat{x}_0^{(j)}(k), \hat{x}_l^{(j)}(k), w_{i0}^{(j)}(k), w_{il}^{(j)}(k), u_j(k), \Delta t \right\}$$

Roll back

Repair step after the trust detector has already found bad contributors

At every time step k , before the observer update is finalized, we save the information needed to replay the update later.

$$\mathcal{B}(k) = \left\{ \hat{x}_i^{(j)}(k), \hat{x}_0^{(j)}(k), \hat{x}_l^{(j)}(k), w_{i0}^{(j)}(k), w_{il}^{(j)}(k), u_j(k), \Delta t \right\}$$

When a malicious set $\mathcal{M}(t)$ is detected at time t , choose the oldest saved :

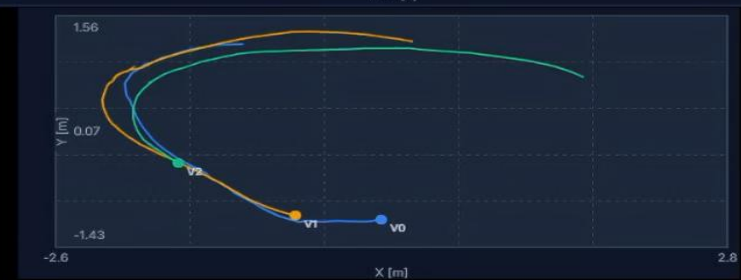
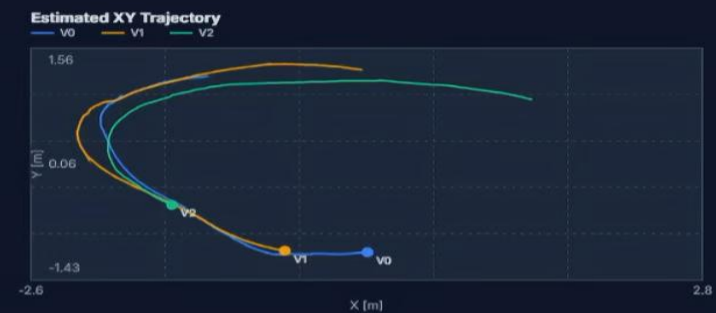
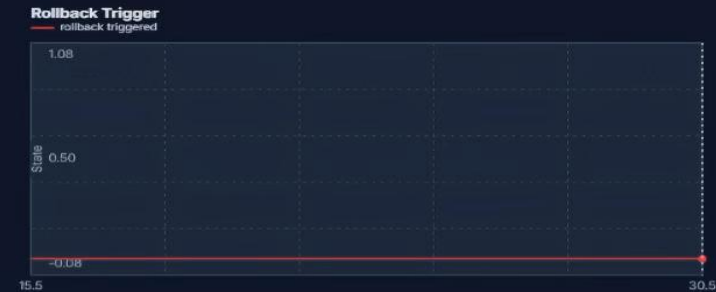
$$\hat{x}_{i,\text{rb}}^{(j)}(k_0) = \hat{x}_{i,\text{save}}^{(j)}(k_0) \quad k_0 = t - K$$

Then replay each step $k = k_0, \dots, t - 1$, excluding malicious sources:

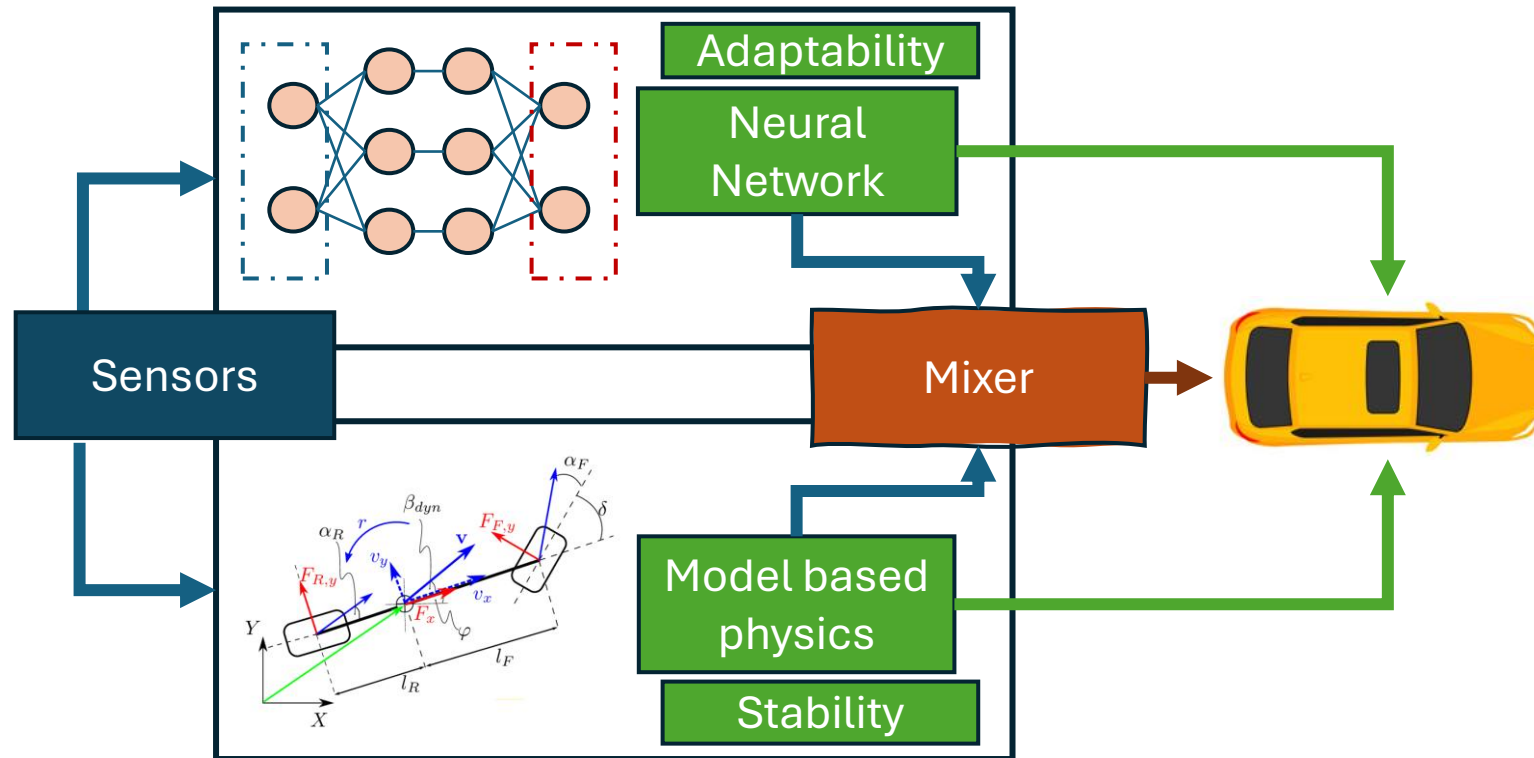
$$z_{i,\text{rb}}^{(j)}(k) = \hat{x}_{i,\text{rb}}^{(j)}(k) + \mathbf{1}_{0 \notin \mathcal{M}} w_{i0}^{(j)}(k) \left(\hat{x}_0^{(j)}(k) - \hat{x}_{i,\text{rb}}^{(j)}(k) \right) + \sum_{l \in \mathcal{N}_i \setminus \mathcal{M}} w_{il}^{(j)}(k) \left(\hat{x}_l^{(j)}(k) - \hat{x}_{i,\text{rb}}^{(j)}(k) \right)$$
$$\hat{x}_{i,\text{rb}}^{(j)}(k+1) = f \left(z_{i,\text{rb}}^{(j)}(k), u_j(k), \Delta t \right)$$

Final replacement:

$$\hat{x}_i^{(j)}(t) \leftarrow \hat{x}_{i,\text{rb}}^{(j)}(t)$$



Robust KalmanNet State Estimation



Problem Setup: Signals and Measurement

Measurement vector

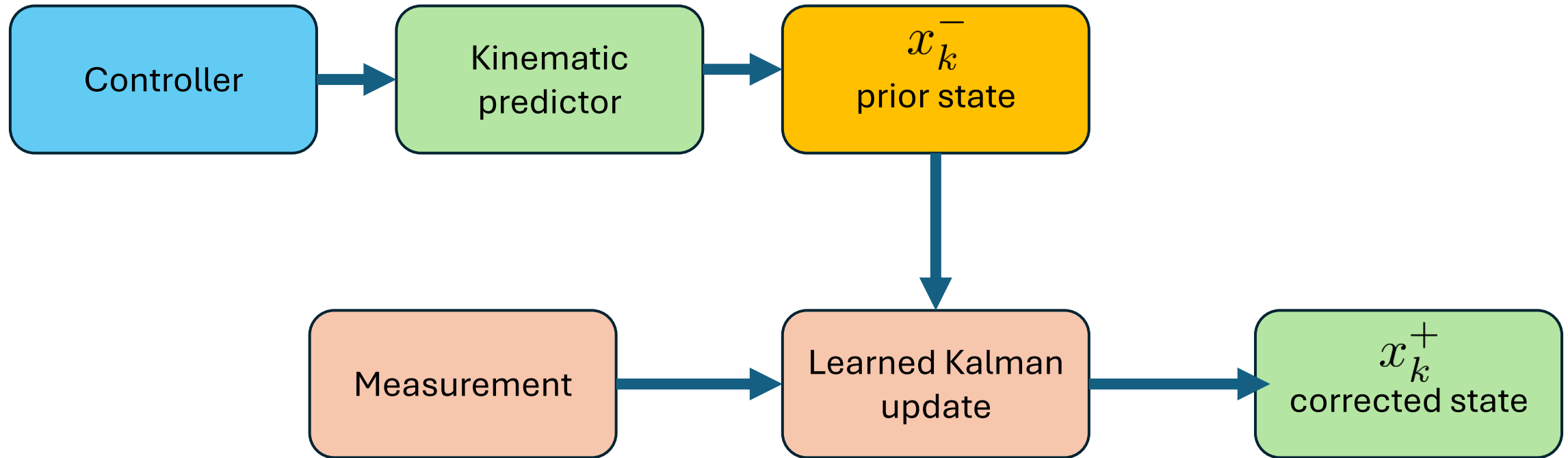
$$z_k = \begin{bmatrix} x_{gps,k} & y_{gps,k} & \psi_k & v_k \end{bmatrix}^T$$

When **attack in the sensor layer** , its will be affected the performance of the state estimator .

Break down all the chain of estimation before

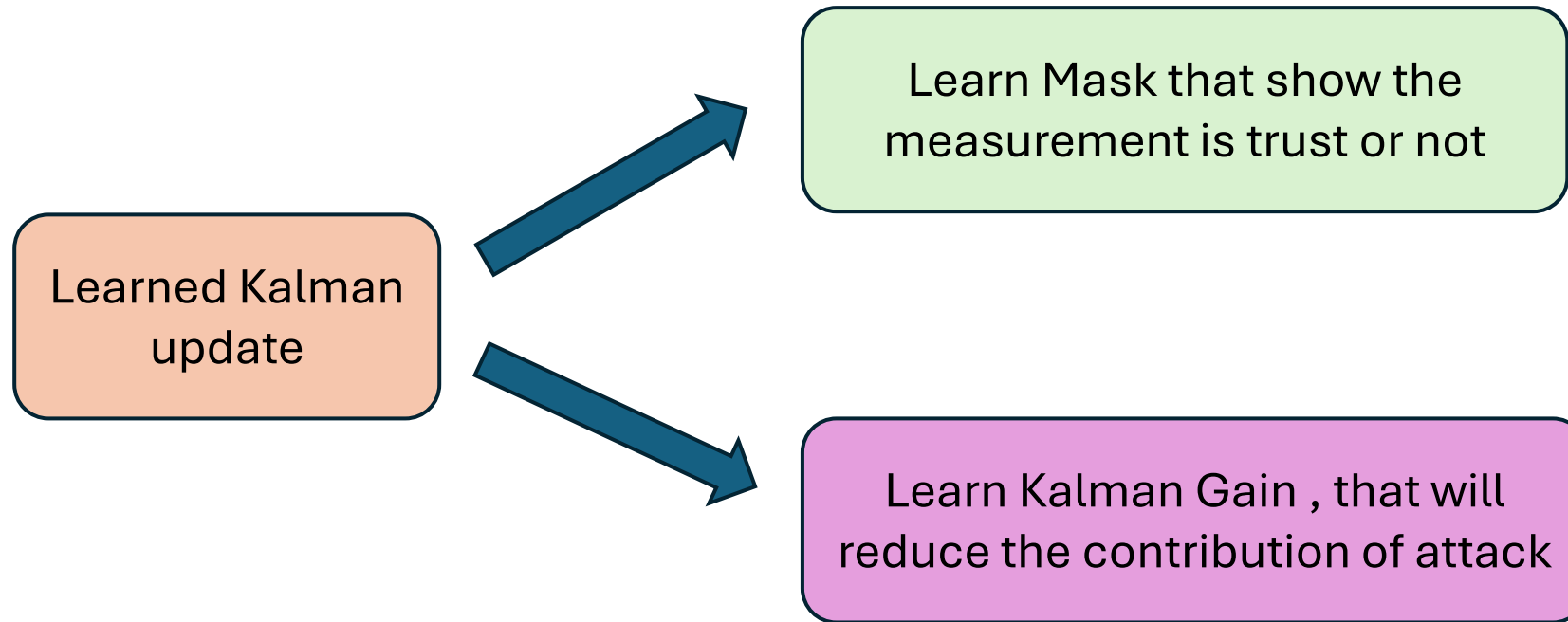
RKNet keep the performance while rejecting corrupted GPS and velocity measurements.

RKNet Concept Design



- Prediction is physics-based.
- Update learns measurement reliability mask and Kalman gain.
- Main robustness target: GPS position and velocity corruption.

RKNet Concept Design



- Prediction is physics-based.
- Update learns measurement reliability mask and Kalman gain.
- Main robustness target: GPS position and velocity corruption.

Prediction Math: Bicycle Model

$$x_k^- = x_{k-1}^+ + v_k \cos(\psi_{k-1}^+) \Delta t$$

$$y_k^- = y_{k-1}^+ + v_k \sin(\psi_{k-1}^+) \Delta t$$

$$\psi_k^- = \psi_{k-1}^+ + \frac{v_k \tan(\delta_k)}{L} \Delta t$$

$$v_k^- = v_{k-1}^+ + a_k \Delta t$$

$$a_k = \frac{-v_{k-1}^+ + u_k}{\tau}$$

Config values: $L = 0.2$, $\tau = 0.301$, $g = 6.598$, $|v| \leq 2.0$.

Learn Robust Update Math

1. Innovation

$$\mathbf{r}_k = \mathbf{z}_k - H\mathbf{x}_k^-$$

2. Prediction difference

$$\Delta\mathbf{x}_k = \mathbf{x}_k^- - \mathbf{x}_{k-1}^+$$

3. Measurement difference

$$\Delta\mathbf{z}_k = \mathbf{z}_k - \mathbf{z}_{k-1}$$

4. Measurement availability

$$a_k = [\textit{gps}_{valid}, \textit{gps}_{valid}, \theta_{valid}, 1]$$

Learned Robust Update Math

1. Innovation

$$\mathbf{r}_k = \mathbf{z}_k - H\mathbf{x}_k^-$$

2. Prediction difference

$$\Delta\mathbf{x}_k = \mathbf{x}_k^- - \mathbf{x}_{k-1}^+$$

3. Measurement difference

$$\Delta\mathbf{z}_k = \mathbf{z}_k - \mathbf{z}_{k-1}$$

4. Measurement availability

$$\mathbf{a}_k = [\textit{gps}_{valid}, \textit{gps}_{valid}, \theta_{valid}, 1]$$

Updater features

$$\mathbf{f}_k = [\Delta\mathbf{x}_k, \mathbf{r}_k, \Delta\mathbf{z}_k, \mathbf{a}_k]$$

Measurement mask



$$\mathbf{m}_k^z = \sigma(MLP(\mathbf{f}_k))$$

Learned Robust Update Math

1. Innovation

$$\mathbf{r}_k = \mathbf{z}_k - H\mathbf{x}_k^-$$

2. Prediction difference

$$\Delta\mathbf{x}_k = \mathbf{x}_k^- - \mathbf{x}_{k-1}^+$$

3. Measurement difference

$$\Delta\mathbf{z}_k = \mathbf{z}_k - \mathbf{z}_{k-1}$$

4. Measurement availability

$$\mathbf{a}_k = [\textit{gps}_{\textit{valid}}, \textit{gps}_{\textit{valid}}, \theta_{\textit{valid}}, 1]$$

Updater features

$$\mathbf{f}_k = [\Delta\mathbf{x}_k, \mathbf{r}_k, \Delta\mathbf{z}_k, \mathbf{a}_k]$$

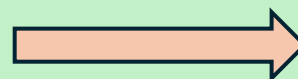
Measurement mask

$$\mathbf{m}_k^z = \sigma(\textit{MLP}(\mathbf{f}_k))$$



Learned Kalman gain

$$\mathbf{h}_k = \textit{GRU}(\mathbf{m}_k^z \odot \mathbf{f}_k, \mathbf{h}_{k-1})$$



$$K_k = \tanh(\textit{MLP}(\mathbf{h}_k))$$

Learned Robust Update Math

1. Innovation

$$\mathbf{r}_k = \mathbf{z}_k - H\mathbf{x}_k^-$$

2. Prediction difference

$$\Delta\mathbf{x}_k = \mathbf{x}_k^- - \mathbf{x}_{k-1}^+$$

3. Measurement difference

$$\Delta\mathbf{z}_k = \mathbf{z}_k - \mathbf{z}_{k-1}$$

4. Measurement availability

$$\mathbf{a}_k = [\textit{gps}_{\textit{valid}}, \textit{gps}_{\textit{valid}}, \theta_{\textit{valid}}, 1]$$

Updater features

$$\mathbf{f}_k = [\Delta\mathbf{x}_k, \mathbf{r}_k, \Delta\mathbf{z}_k, \mathbf{a}_k]$$

Measurement mask

$$\mathbf{m}_k^z = \sigma(\textit{MLP}(\mathbf{f}_k))$$

Learned Kalman gain

$$\mathbf{h}_k = \textit{GRU}(\mathbf{m}_k^z \odot \mathbf{f}_k, \mathbf{h}_{k-1})$$

$$K_k = \tanh(\textit{MLP}(\mathbf{h}_k))$$

Final update

$$\mathbf{x}_k^+ = \mathbf{x}_k^- + K_k \tilde{\mathbf{r}}_k \quad \text{where} \quad \tilde{\mathbf{r}}_k = \mathbf{m}_k^z \odot \mathbf{r}_k$$

Attack Setup: Focus on GPS and Velocity

GPS position attacks

- Noise: $z_{xy}^{\text{atk}} = z_{xy} + \epsilon$
- Jump: $z_{xy}^{\text{atk}} = z_{xy} + \mathbf{b}_{\text{jump}}$
- Freeze: $z_{xy,k}^{\text{atk}} = z_{xy,k_0}, \quad k \geq k_0$
- Dropout: $\text{gps_valid}_k = 0$

Velocity attacks

- Bias: $z_v^{\text{atk}} = z_v + b_v$
- Scale: $z_v^{\text{atk}} = \alpha z_v$
- Freeze: $z_{v,k}^{\text{atk}} = z_{v,k_0}, \quad k \geq k_0$
- Noise: $z_v^{\text{atk}} = z_v + \epsilon_v$
- Zero-out: $z_{v,k}^{\text{atk}} = 0$

Attack labels supervise the mask:

$$\text{label}_{k,i} = \begin{cases} 1, & \text{measurement channel } i \text{ attacked} \\ 0, & \text{clean} \end{cases}$$

$$m_{k,i}^{z,\text{target}} = 1 - \text{label}_{k,i}$$

$$m_{k,i}^{z,\text{target}} = \begin{cases} 1, & \text{trust measurement channel } i \\ 0, & \text{suppress measurement channel} \end{cases}$$

Saved fields: raw sensors, GPS flags, z_k , target $\mathbf{x}_k^*$, timestamps.

Training schedule

- Phase A : Train updater only. Train updater with good previous state input
- Phase B : End-to-end fine tuning. Model uses its own previous output train the full recurrent estimation loop
- Validation split: 20% tail.

Loss

$$\mathcal{L} = \lambda_u L_{upd} + \lambda_p L_{pred} + \lambda_m L_{mask} + \lambda_K L_{gain} + \lambda_{\Delta K} L_{\Delta K} + \lambda_{\Delta m} L_{\Delta m}$$

$$L_{upd} = MSE_W(\mathbf{x}^+, \mathbf{x}^*), \quad L_{pred} = MSE_W(\mathbf{x}^-, \mathbf{x}^*)$$

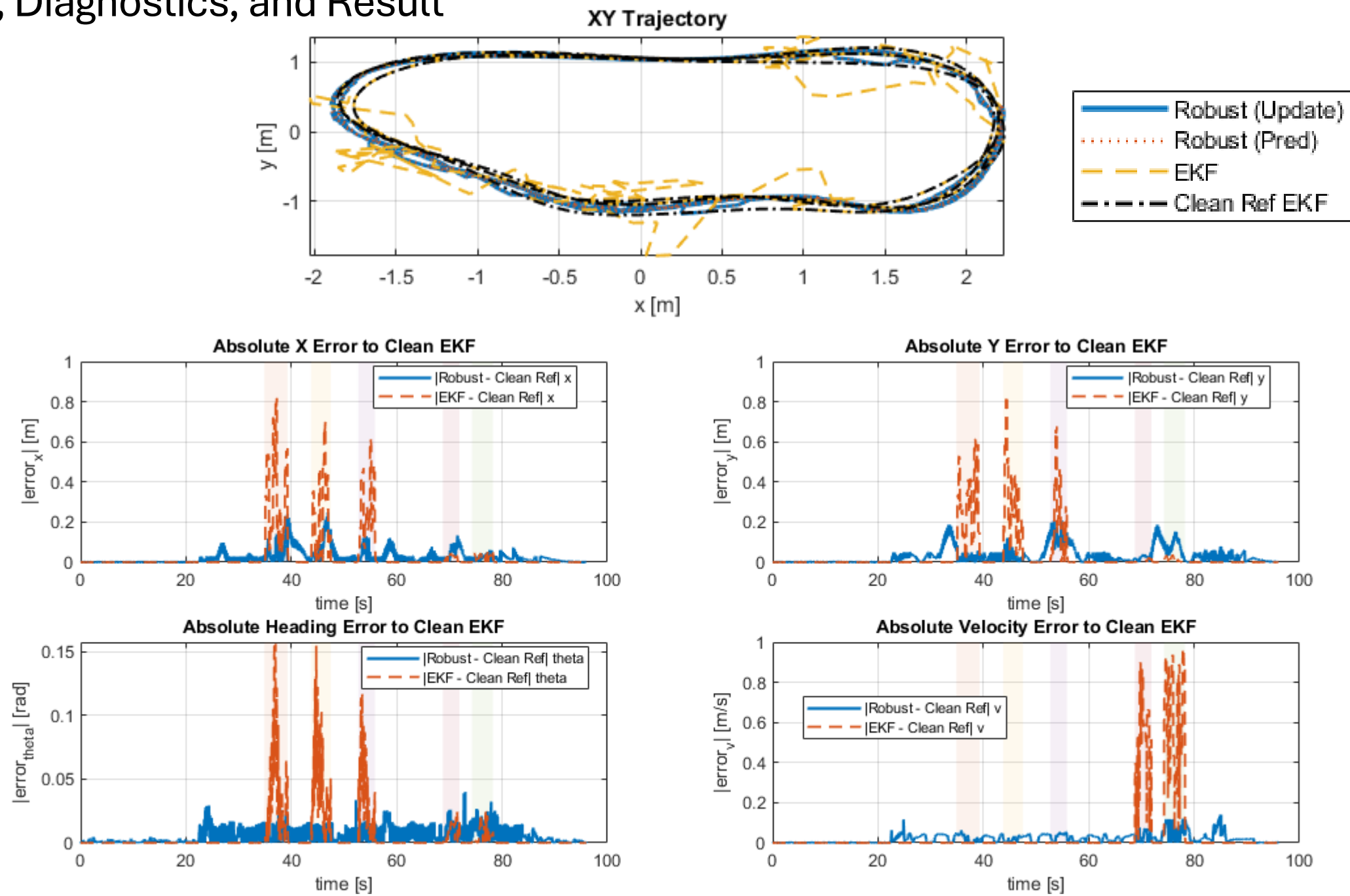
$$L_{mask} = BCE(\mathbf{m}^z, \mathbf{m}^{z,target}), \quad L_{gain} = \|K\|_2^2$$

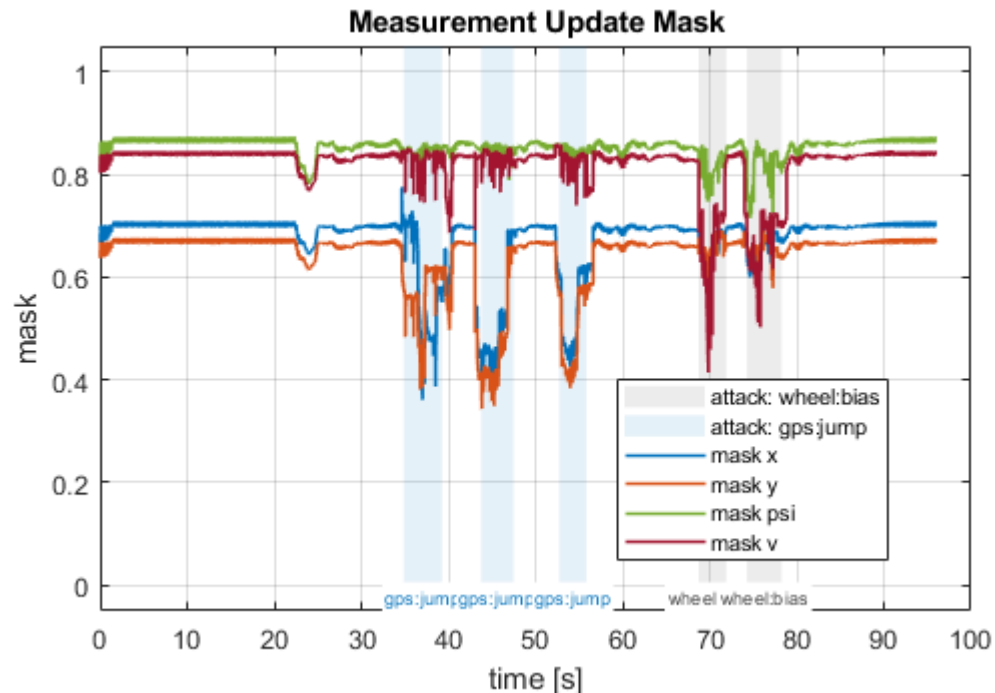
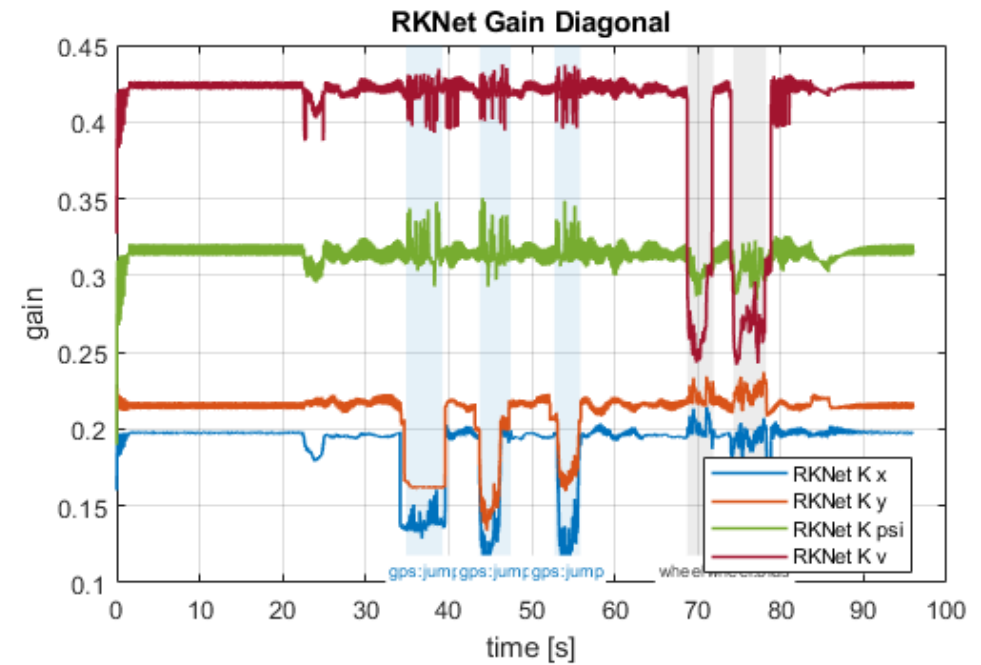
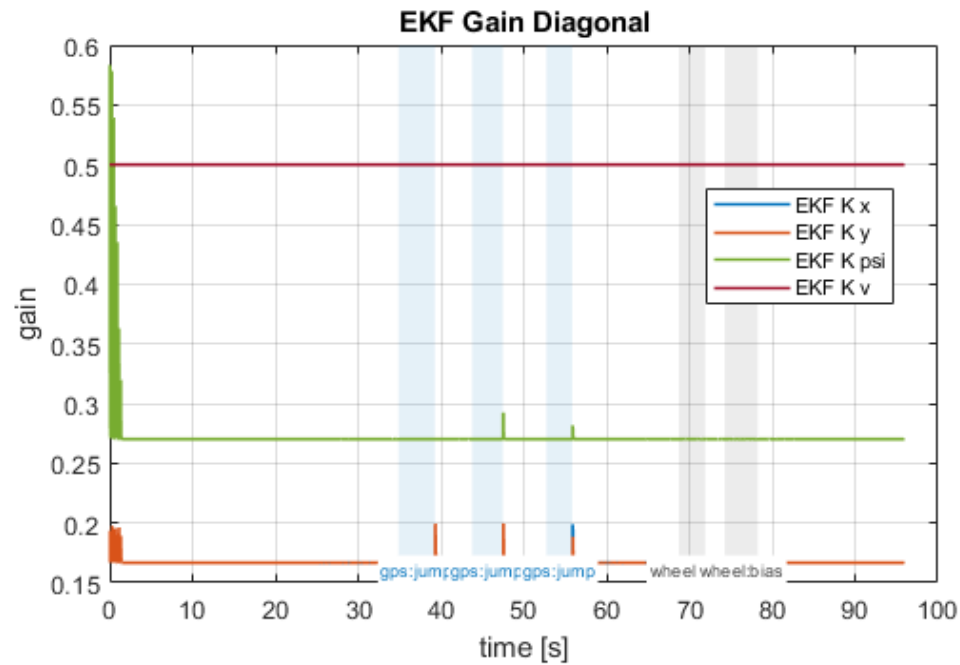
$$L_{\Delta K} = \|K_k - K_{k-1}\|_2^2, \quad L_{\Delta m} = \|\mathbf{m}_k^z - \mathbf{m}_{k-1}^z\|_2^2$$

- L_{upd} : final corrected state should match the reference.
- L_{pred} : prior prediction should stay physically reasonable.
- L_{mask} : clean channel target = 1, attacked channel target = 0.
- L_{gain} : avoid very large Kalman gains.
- $L_{\Delta K}$: avoid gain chatter.
- $L_{\Delta m}$: avoid mask flicker.

$$\lambda_u = 0.85, \lambda_p = 0.0, W = [1.0, 1.0, 2.5, 1.2], \lambda_K = 0.01, \lambda_{\Delta K} = 0.02.$$

Testing, Diagnostics, and Result





Mask and Gain RKNet go down ↓

So less depend on the measurement ,
more robust